Lesson Plan: Module 1

Radio Frequency Module Selection, Hardware Bring-Up, and Cooperative Target–Interceptor Subsystem Design

---

An Educational Framework for AI-Driven Signal Processing

Student:	Jude Eschete
Project Advisor:	Dr. Bernard Yett
Program:	M.S. Computer Engineering
Institution:	Stevens Institute of Technology
Date Range:	February 28 – March 14, 2026

This document presents a lesson plan for the first module of the capstone project, covering Radio Frequency transceiver selection, hardware platform validation, and the design of the cooperative-target / interceptor subsystem that forms the foundation for all subsequent Field-Programmable Gate Array processing and Machine Learning work. Two STM32+RFM95W stations exchange DATA and ACK traffic on a shared 906.5 MHz channel; a third STM32+RFM95W interceptor ("Threat") passively captures the bidirectional traffic, forwards it to the FPGA, and operates a button-gated PAUSE protocol that targets the highest-RSSI station — closing the intercept-and-act loop without violating FCC ISM-band rules.

JEschete@stevens.edu

Contents

1 Module Overview **1**
- 1.1 Context Within the Project . . . 1
- 1.2 Prerequisites . . . 1

2 Session Schedule **3**

3 Session 1: RF Module Trade Study **4**
- 3.1 Objectives . . . 4
- 3.2 Selection Criteria . . . 4
- 3.3 Candidate Modules . . . 5
- 3.4 Link Budget Analysis . . . 5

4 Session 2: Hardware Bring-Up and Validation **6**
- 4.1 Objectives . . . 6
- 4.2 STM32 Firmware Bring-Up . . . 6
  - 4.2.1 Development Environment . . . 6
  - 4.2.2 SPI Configuration . . . 7
  - 4.2.3 Validation Steps . . . 8
- 4.3 FPGA Toolchain Validation . . . 8
  - 4.3.1 Development Environment . . . 8
  - 4.3.2 Loopback Design . . . 9

5 Session 3: Cooperative Target Subsystem Design **10**
- 5.1 Objectives . . . 10
- 5.2 Cooperative Target Architecture Overview . . . 10
- 5.3 Packet Format Design . . . 11
- 5.4 Transmission Scheduling and Bidirectional Behavior . . . 13
  - 5.4.1 Initiator Scheduling Modes . . . 13
  - 5.4.2 Responder Behavior . . . 13
  - 5.4.3 PAUSE Listener Behavior . . . 13
  - 5.4.4 Timing Implementation . . . 13
- 5.5 Cooperative Target Firmware Architecture . . . 14
- 5.6 Validation: Two-Target Ping-Pong with PAUSE Behavioral Response . . . 15

6 Session 4: Interceptor (Threat) Subsystem Design **17**
- 6.1 Objectives . . . 17
- 6.2 Interceptor Architecture . . . 17
- 6.3 Interceptor-to-FPGA Interface . . . 18
  - 6.3.1 Interface Selection . . . 18
  - 6.3.2 Interceptor-to-FPGA Frame Format . . . 18
- 6.4 Interceptor Firmware Architecture . . . 19
- 6.5 FPGA Ingestion Interface . . . 19
- 6.6 System Architecture Diagram . . . 20

7 Hardware Platform Reference **22**

8 Assessment Rubric **22**

9 Recommended Resources **23**
9.1 Datasheets and Technical References 23
9.2 Application Notes 23
9.3 Textbooks 23

10 Transition to Module 2 **24**

Glossary **25**

1 Module Overview

Module 1 Learning Objectives

Upon completion of this module, the student (or future learner following this curriculum) will be able to:

LO-1 Evaluate commercially available Radio Frequency (RF) transceiver modules against project-specific requirements and justify a final selection with quantitative criteria.

LO-2 Verify end-to-end hardware communication between the STMicroelectronics STM32 Nucleo-L476RG microcontroller and the selected RF module using basic firmware.

LO-3 Confirm Field-Programmable Gate Array (FPGA) toolchain readiness on the Digilent Nexys A7-100T and validate a minimal loopback design.

LO-4 Design a complete RF cooperative-target and interceptor subsystem architecture, including bidirectional DATA/ACK traffic, transmission scheduling, packet formatting, the button-gated PAUSE protocol, and the interceptor-to-FPGA data flow.

LO-5 Produce a hardware requirements document and subsystem design specification that downstream modules can build upon directly.

1.1 Context Within the Project

This module spans the first two milestones in the project timeline. It establishes the hardware baseline and subsystem architecture upon which all subsequent modules depend. The RF module trade study and hardware bring-up validate that the chosen components can communicate reliably, while the cooperative-target / interceptor subsystem design defines the architecture that the FPGA register-transfer level (Register-Transfer Level (RTL)) design (Module 2) and Machine Learning (ML) pipeline (Module 3) will consume.

1.2 Prerequisites

Students beginning this module should have the background listed below. For students who need to catch up on any item, a recommended primer is noted; all cited works appear in full in Section 9.

- Familiarity with C/C++ embedded programming (General-Purpose Input/Output (GPIO), Serial Peripheral Interface (SPI), Universal Asynchronous Receiver/Transmitter (UART), and Inter-Integrated Circuit (I^2C)) — see Valvano, *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers*, for peripheral fundamentals, and the STM32L476xx Reference Manual (RM0351) for the specific peripheral registers used in this module.

- Basic understanding of digital modulation (Amplitude Shift Keying (ASK), Frequency Shift Keying (FSK), and LoRa Chirp Spread Spectrum (CSS) concepts) — Sklar, *Digital Communications: Fundamentals and Applications*, covers the modulation theory; Semtech Application Note AN1200.22, "LoRa Modulation Basics," is the concise primer for LoRa-specific CSS.

- Experience with VHSIC Hardware Description Language (VHDL) and the FPGA synthesis flow — Chu, *FPGA Prototyping by VHDL Examples: Xilinx Artix-7 Edition*, is the pri-

mary recommended text; Haskell and Hanna, *Digital Design Using Digilent FPGA Boards: VHDL / Vivado Edition*, provides a Nexys-board-specific walkthrough of the Vivado toolchain.

- Introductory exposure to machine learning (supervised classification) — a formal reference is introduced in Module 4; any standard introductory text on supervised classification is sufficient background here.
- Access to the hardware listed in Section 7

2 Session Schedule

The module spans four sessions across two project milestones. Each session has a thematic focus, concrete tasks, and a deliverable checkpoint.

Table 1: Module 1 session breakdown.

Session	Focus Area	Key Activities
1	RF Module Trade Study	Define selection criteria; evaluate candidate modules (RFM95W, SX1276, Ra-02, E32); perform link budget analysis; select and procure final module.
2	Hardware Bring-Up	Flash STM32 with SPI driver firmware; verify RF module register access; transmit/receive basic packets; confirm Nexys A7 toolchain and loopback; produce hardware requirements document.
3	Cooperative Target Subsystem Design	Define cooperative-target transmission architecture (Target A initiates DATA, Target B replies with ACK); design 32-byte packet format with packed DATA/ACK/PAUSE type field; implement configurable transmission scheduling and the PAUSE listener path on STM32; validate two-target ping-pong operation.
4	Interceptor (Threat) Subsystem Design	Design interceptor data path from RF front-end to FPGA input; implement passive capture of every DATA, ACK, and PAUSE frame on the channel; implement the per-Beacon-ID RSSI table and the button-gated PAUSE scheduler that targets the highest-RSSI station; define FPGA ingestion interface; validate end-to-end target-to-FPGA data flow including PAUSE behavioral response on the targets.

3 Session 1: RF Module Trade Study

3.1 Objectives

- Define quantitative and qualitative selection criteria for the RF transceiver module.
- Evaluate at least four candidate modules against those criteria.
- Perform a basic link budget analysis for the benchtop operating environment.
- Produce a justified final selection with procurement plan.

3.2 Selection Criteria

The following table defines the evaluation dimensions. Each criterion is weighted to reflect its importance to the project's educational and technical goals.

Table 2: RF module selection criteria and weights.

Criterion	Weight	Description
SPI Interface Compatibility	20%	Must support standard SPI (CPOL/CPHA modes) at voltages compatible with the STM32 Nucleo (3.3 V logic).
Frequency Band	15%	915 MHz industrial, scientific, and medical (ISM) band preferred for U.S. regulatory compliance.
Modulation Support	15%	Must support LoRa (CSS) and ideally FSK/on-off keying (OOK) for multi-modulation classification exercises.
Documentation & Community	15%	Availability of datasheets, application notes, open-source driver libraries, and community forums.
Cost & Availability	10%	Per-unit cost and lead time; must be procurable from standard distributors (Digi-Key, Mouser, Adafruit).
Power Consumption	10%	Low enough for USB-powered benchtop operation; transmit current < 120 mA.
Packet Engine Features	10%	On-chip first in, first out (FIFO) buffering, cyclic redundancy check (CRC), and packet handling to simplify initial firmware development.
Physical Form Factor	5%	Breadboard- or breakout-board-friendly; no fine-pitch ball grid array (BGA) requiring reflow.

3.3 Candidate Modules

Table 3: Candidate RF transceiver modules.

Module	Chipset	Freq.	Notes
Adafruit RFM95W	SX1276	915 MHz	Breadboard-ready breakout; extensive tutorials.
AI-Thinker Ra-02	SX1278	433 MHz	Low cost; 433 MHz limits U.S. ISM exercises.
EByte E32-915T20D	SX1276	915 MHz	UART interface (not SPI); 100 mW TX power.
HopeRF RFM96W	SX1276	915 MHz	SMD module; needs custom breakout or adapter.

3.4 Link Budget Analysis

For the benchtop environment (transmitter–receiver separation $\leq 10\,\mathrm{m}$ indoors), students should compute:

$$P_{\mathrm{rx}} = P_{\mathrm{tx}} + G_{\mathrm{tx}} + G_{\mathrm{rx}} - L_{\mathrm{path}} - L_{\mathrm{misc}} \tag{1}$$

where:

- P_{tx}: Transmit power (dBm), typically +13 to +20 dBm for SX1276-based modules.
- $G_{\mathrm{tx}}, G_{\mathrm{rx}}$: Antenna gains (dBi); assume $\approx 2\,\mathrm{dBi}$ for standard whip antennas.
- L_{path}: Free-space path loss at 915 MHz over distance d:

$$L_{\mathrm{path}} = 20\log_{10}(d) + 20\log_{10}(f) - 147.55 \quad [\mathrm{dB}] \tag{2}$$

- L_{misc}: Miscellaneous losses (cable, mismatch, indoor multipath margin $\approx 10\,\mathrm{dB}$).

Exercise: Link Budget Worksheet

Calculate the received power for the following scenario:

- $P_{\mathrm{tx}} = +14\,\mathrm{dBm}$, $G_{\mathrm{tx}} = G_{\mathrm{rx}} = 2\,\mathrm{dBi}$
- $d = 5\,\mathrm{m}$, $f = 915\,\mathrm{MHz}$
- $L_{\mathrm{misc}} = 10\,\mathrm{dB}$ (indoor margin)

Verify that P_{rx} exceeds the SX1276 receiver sensitivity ($-137\,\mathrm{dBm}$ at SF12, 125 kHz BW) by a comfortable margin. Document the link margin and discuss its implications for multi-beacon scaling.

Session 1 Deliverable

RF Module Trade Study Report — A 3–5 page document containing the weighted evaluation matrix, link budget calculations, final module selection with justification, and a procurement plan (vendor, cost, estimated delivery).

4 Session 2: Hardware Bring-Up and Validation

4.1 Objectives

- Establish SPI communication between the STM32 Nucleo-L476RG and the selected RF module.
- Transmit and receive basic test packets between two beacon nodes.
- Validate the Nexys A7-100T FPGA toolchain (Vivado) with a minimal loopback design.
- Document all hardware interfaces, pin assignments, and power requirements.

4.2 STM32 Firmware Bring-Up

4.2.1 Development Environment

Two supported workflows are documented. Students may choose either; the course reference environment is the VS Code flow because it unifies the STM32, serial monitor, and (later) documentation/version-control workflow in a single tool.

Option A — VS Code with the official STMicroelectronics extension stack (recommended). Install the *STM32 VS Code Extension* from the Visual Studio Code marketplace. It is the official ST-maintained port of the STM32CubeIDE toolchain and automatically installs the supporting sub-extensions listed below. No separate CubeIDE installation is required.

- **Umbrella extension:** `stmicroelectronics.stm32-vscode-extension` (pulls the full sub-extension set).
- **Project management and build:** `stm32cube-ide-project-manager`, `stm32cube-ide-build-cmake`, `stm32cube-ide-build-analyzer`, and `stm32cube-ide-bundles-manager` (auto-fetches the ARM GCC toolchain and Hardware Abstraction Layer (HAL) packs).
- **IntelliSense:** `stm32cube-ide-clangd` (pre-configured for ARM cross-compilation).
- **Debug:** `stm32cube-ide-debug-core` with the `debug-stlink-gdbserver` back end (the Nucleo-L476RG's on-board ST-LINK/V2.1 is supported out of the box; `debug-jlink-gdbserver` and `debug-generic-gdbserver` are also installed for external probes).
- **Peripheral register view:** `stm32cube-ide-registers` for live register inspection during debug.
- **RTOS awareness:** `stm32cube-ide-rtos` (unused for the Module 1 bare-metal firmware; retained for later modules if an RTOS is introduced).

- **Supporting general extensions:**
  - `ms-vscode.cpptools` (+ extension pack) — fallback C/C++ language server.
  - `ms-vscode.cmake-tools` — CMake integration.
  - `marus25.cortex-debug` — alternate Cortex-M debugger with SVD, RTT, and ITM support.
  - `dan-c-underwood.arm` — ARM assembly syntax highlighting.
  - `eclipse-cdt.serial-monitor` — UART monitor for the Nucleo's virtual COM port directly inside VS Code.

With this stack installed, the full workflow — peripheral configuration (integrated CubeMX), code generation, build, flash over the on-board ST-LINK, single-step debug, peripheral register inspection, and UART monitoring — runs entirely inside VS Code with no additional tools required.

Option B — Standalone STM32CubeIDE. The classic Eclipse-based STM32CubeIDE (free from ST) remains fully supported. It bundles the HAL libraries and STM32CubeMX code generation in a single installer and may be preferred by students already familiar with Eclipse or those who want CubeMX as a dedicated GUI application.

Optional — STM32Cube Command Line Tools (STM32Cube) (CLT). The STM32Cube Command Line Tools package is a single installer that places the ARM GCC toolchain (`arm-none-eabi-gcc`, `-gdb`, `-objcopy`, . . .), CMake, Ninja, `OpenOCD`, the ST-LINK GDB server, and `STM32_Programmer_CLI` on the system `PATH`. It is not required for the in-lab flow — both options above handle build, flash, and debug entirely through their own integrated tooling — but installing it is recommended if you intend to (a) script builds or flashing from a shell or CI pipeline, (b) run `arm-none-eabi-gdb` or `OpenOCD` outside of VS Code, or (c) keep a single toolchain install that survives Integrated Development Environment (IDE) upgrades. The official STM32 VS Code extension auto-discovers a CLT installation when present and prefers it over the Bundles Manager download.

Common target details (all options).

- **Target Microcontroller Unit (MCU):** STM32L476RG (ARM Cortex-M4, 80 MHz, low-power).
- **Debug Interface:** On-board ST-LINK/V2.1 (USB) — no external programmer required.
- **Serial console:** Nucleo virtual COM port via the same ST-LINK USB connection (used for debug prints and Received Signal Strength Indicator (RSSI)/Signal-to-Noise Ratio (SNR) logging).

4.2.2 SPI Configuration

Using the integrated STM32CubeMX peripheral configurator (available from either the VS Code extension or standalone STM32CubeIDE — see Section 4.2.1), configure the SPI peripheral as follows:

Table 4: SPI peripheral configuration for RF module communication.

Parameter	Setting	Rationale
Mode	Full-Duplex Master	MCU drives clock to RF module
Clock Polarity (CPOL)	0	SX1276 default
Clock Phase (CPHA)	0	SX1276 default
Data Size	8 bits	Register-level access
Baud Rate	$\leq 10\,\text{MHz}$	SX1276 SPI max = 10 MHz
NSS	Software (GPIO)	Manual chip select control

4.2.3 Validation Steps

1. **Register Read Test:** Read the SX1276 `RegVersion` register (address `0x42`). Expected return value: `0x12`. This confirms SPI wiring and clock configuration.
2. **Register Write/Read-Back:** Write a known value to a writable register (e.g., `RegFrfMsb`), read it back, and verify.
3. **Mode Transition:** Command the module from `SLEEP` → `STANDBY` → `TX` and verify the mode bits in `RegOpMode`.
4. **Packet Transmission:** Configure two STM32+RF nodes. Transmit a known payload from Node A; verify reception and CRC pass on Node B. Log RSSI and SNR values.

Common Pitfalls

- **Level mismatch:** Ensure the RF module operates at 3.3 V logic. The Nucleo's Arduino-compatible headers provide 3.3 V by default, but verify with a multimeter before connecting.
- **SPI bus contention:** If other SPI peripherals share the bus, ensure proper chip-select management.
- **Antenna:** Always connect an antenna (or 50 Ω dummy load) before transmitting to avoid damage to the RF front-end.
- **Reset sequencing:** The SX1276 requires a hardware reset pulse (active low, $\geq 100\,\mu$s) after power-on before SPI access is reliable.

4.3 FPGA Toolchain Validation

4.3.1 Development Environment

- **Tool:** AMD Vivado Design Suite (WebPACK edition, free for Artix-7).
- **Board:** Digilent Nexys A7-100T (XC7A100T-1CSG324C).
- **Constraint File:** Use the Digilent-provided `Nexys-A7-100T-Master.xdc`.

4.3.2 Loopback Design

Implement a minimal design to validate the toolchain and board:

1. **LED Echo:** Map the 16 slide switches to the 16 LEDs (combinational pass-through). Confirms pin constraints and bitstream generation.
2. **UART Loopback:** Instantiate a simple UART receiver and transmitter at 115200 baud on the USB-UART bridge. Echo received characters back to the host terminal. Confirms clocking, UART IP, and host communication.
3. **Clock Verification:** Use an Integrated Logic Analyzer (ILA) (Integrated Logic Analyzer) core to probe a free-running counter driven by the 100 MHz oscillator. Verify the expected count rate.

Session 2 Deliverable

Hardware Requirements Document — A formal document capturing:

1. **Bill of Materials (Bill of Materials (BOM)):** Part numbers, quantities, unit costs, vendors, and lead times for all components.
2. **Interface Specification:** Pin mappings for STM32↔RF module (SPI + GPIO), planned FPGA↔receiver interface, UART debug connections.
3. **Power Budget:** Measured current draw for each subsystem; confirm USB power sufficiency.
4. **Validation Evidence:** Screenshots or logic analyzer captures of successful SPI transactions, packet reception logs with RSSI/SNR, FPGA loopback test results.
5. **Risk Register:** Identified risks (e.g., RF interference in lab environment, component lead times) with mitigation strategies.

5 Session 3: Cooperative Target Subsystem Design

5.1 Objectives

- Define the role and behavior of each cooperative-target node within the system.
- Design a packet format that carries the metadata required for downstream classification *and* encodes the DATA / ACK / PAUSE packet type without modifying the 32-byte radio frame layout.
- Implement bidirectional firmware: Target A initiates DATA transmissions; Target B replies with ACK; both listen continuously for PAUSE commands addressed to their own Beacon ID.
- Validate two-target ping-pong operation including the PAUSE behavioral response (target prints `"I'm Jammed"` on USART2 while held in PAUSE).

5.2 Cooperative Target Architecture Overview

Each cooperative target consists of an STM32 Nucleo-L476RG paired with an RFM95W LoRa transceiver. Both targets are TX+RX-capable and operate as a half-duplex pair: Target A (Beacon ID `0x01`) initiates DATA on a configurable schedule and listens briefly for an ACK from Target B (Beacon ID `0x02`); Target B normally listens, replies with ACK on every successfully received DATA, and may optionally initiate its own DATA on its own schedule. Both targets continuously monitor the channel for PAUSE commands from the interceptor (Threat, Beacon ID `0xFF`) addressed to their own ID. The cooperative-target subsystem must scale from a single initiator (Target A only, no responder) up to two initiators (Target A and Target B both initiating) plus interleaved PAUSE captures, progressively increasing the demands on the interceptor and FPGA.

Design Principle: Bidirectional but Half-Duplex

The cooperative-target subsystem is intentionally bidirectional — the third radio in the system (the interceptor / Threat) is hardware-symmetric with the targets, but the targets themselves never communicate with the Threat directly. Instead, the Threat passively captures every DATA and ACK frame on the channel and, when its user button is held, emits PAUSE commands on the same channel; targets that receive a PAUSE addressed to their own ID enter a 500 ms transmit hold-off. Each radio is half-duplex (it cannot transmit and receive simultaneously); the firmware design must explicitly switch the RFM95W between `TX`, `STANDBY`, and `RXCONTINUOUS` modes around every operation. The half-duplex constraint is itself a teaching point: students discover that an ACK that arrives before the originator switches back into RX is silently lost, and the system-level fix is a deterministic post-TX RX-window timeout rather than a tighter ACK schedule.

5.3 Packet Format Design

All three packet types (DATA, ACK, PAUSE) share a single fixed-length 32-byte radio frame layout. The packet type is encoded in the low two bits of the `Modulation Code` byte (offset 6); the modulation kind — LoRa, FSK, or OOK — occupies the upper six bits. This packing keeps the FPGA ingestion pipeline (Module 2) and the forwarding-frame layout (Session 4 below) unchanged across all packet types: a downstream parser sees a 32-byte payload regardless of whether the packet is DATA, ACK, or PAUSE.

Table 5: Radio packet format (fixed-length, 32 bytes; same layout for DATA, ACK, PAUSE).

Field	**Offset**	**Size**	**Description**
Preamble Sync	0	2 B	Fixed sync word (`0xAE5A`) for frame detection.
Beacon ID (sender)	2	1 B	Sender ID. Target A = `0x01`, Target B = `0x02`, Threat = `0xFF`.
Sequence Number	3	2 B	Monotonically increasing per-sender. For ACK, echoes the acknowledged DATA sequence.
TX Power	5	1 B	Transmit power setting (dBm), encoded as unsigned integer.
Modulation Code	6	1 B	Packed: bits [7:2] = modulation kind (`0x01`=LoRa, `0x02`=FSK, `0x03`=OOK, each shifted left by 2); bits [1:0] = packet type (`00`=DATA, `01`=ACK, `10`=PAUSE, `11`=reserved).
Spreading Factor	7	1 B	LoRa SF value (7–12); 0x00 if non-LoRa modulation.
Bandwidth Code	8	1 B	Encoded BW setting (e.g., 0x00=125 kHz, 0x01=250 kHz).
Timestamp	9	4 B	STM32 system tick (ms) at time of transmission.
Payload (DATA/ACK) or PAUSE args	13	17 B	DATA/ACK: pseudo-random or structured pattern. PAUSE: byte 0 = target Beacon ID being paused; byte 1 = hold-off duration in 100 ms units; bytes 2–16 = `0xAA` fill.
CRC-16	30	2 B	CRC-16/CCITT-FALSE over bytes 0–29.

Design Rationale: Why Pack the Type into the Modulation Code Byte?

The `Modulation Code` byte already exists in the packet for a different reason — it labels the modulation kind so the downstream classifier has a ground-truth label for the modulation classification task. Packing the 2-bit packet type into the low bits of the same byte avoids changing the 32-byte frame layout (no downstream consumer breaks) and creates a deliberate label-leakage trap that Module 4 students must reason about: a model trained with the raw `Modulation Code` byte as a feature column trivially learns the packet type from the low two bits, even when the task is supposed to be modulation classification only. The Module 4 leakage check (LH4-B) catches this by training a logistic regression on forbidden columns alone and requiring $\geq$95% accuracy — a passing leakage check forces students to mask the low two bits before using `mod_code` as a label.

The 17-byte payload region of DATA and ACK packets carries pseudo-random bit sequences for bit-error-rate (BER) testing or structured patterns that exercise specific demodulator behavior. PAUSE packets reuse the same payload region for two purpose-specific bytes (target ID, hold-off duration) plus `0xAA` fill, keeping the radio frame size constant.

5.4 Transmission Scheduling and Bidirectional Behavior

The cooperative-target subsystem's traffic pattern is determined jointly by Target A's initiator schedule, Target B's responder behavior, and any optional initiator activity from Target B. The scheduling strategy must balance simplicity (suitable for an educational context) with sufficient sophistication to create realistic multi-access scenarios.

5.4.1 Initiator Scheduling Modes

Students implement at least two initiator scheduling modes:

1. **Fixed Interval (TDMA-like):** The initiator emits DATA on a periodic schedule.

$$t_{\text{tx}}^{(k)} = T_{\text{frame}} \cdot n + \Delta t \cdot k, \quad k = 0, 1, \ldots, N-1 \tag{3}$$

   where T_{frame} is the frame period, Δt is the slot duration, k is the initiator index (only meaningful when both targets initiate), and n is the frame counter. This mode produces predictable, non-colliding traffic and is the starting point for initial validation.

2. **Randomized Interval (ALOHA-like):** The initiator emits DATA after a random back-off drawn from a configurable uniform distribution $[T_{\min}, T_{\max}]$. This mode introduces collisions and partial overlaps, creating a more challenging scenario for the FPGA detection and the ML classifier in later modules.

5.4.2 Responder Behavior

Target B's responder path is event-driven, not scheduled: every successfully received DATA addressed (implicitly, by ID context) to the cooperative-target group triggers a single ACK transmission whose `Sequence Number` field echoes the received DATA's sequence. The responder does not retransmit on missed DATA; the originator detects the missed ACK via its own RX-window timeout and proceeds.

5.4.3 PAUSE Listener Behavior

Both targets continuously monitor the channel between scheduled transmissions for PAUSE packets. On receiving a PAUSE whose payload byte 0 matches the target's own Beacon ID, the firmware sets `paused_until_ms = HAL_GetTick() + (payload[1] × 100)`. While `HAL_GetTick()` < `paused_until_ms`: scheduled DATA transmissions are skipped, ACK responses are still permitted (the framework treats ACK as a higher-priority response to a real DATA reception), and the firmware prints `"I'm Jammed"` on USART2 once per pause-entry (throttled, not per-tick). When `paused_until_ms` elapses without further PAUSE refresh, the target resumes normal initiator behavior. PAUSEs addressed to a different ID are received and discarded silently — the firmware does not log them, and they do not affect the receiving target's behavior.

5.4.4 Timing Implementation

Transmission timing is driven by the STM32's hardware timer peripheral (`TIM2` or `TIM6`) configured in periodic mode at a 1 ms tick. Key implementation details:

- Use the STM32 HAL timer interrupt callback (`HAL_TIM_PeriodElapsedCallback`) to maintain a `volatile uint32_t ms_count` that the main-loop Finite State Machine (FSM) polls.

- For randomized initiator mode, seed the pseudo-random number generator with a unique value per target (e.g., Beacon ID XOR'd with an ADC noise sample).
- Log each transmission, ACK, and PAUSE-receive event over USART2 with a millisecond timestamp for ground-truth correlation with the interceptor's captured dataset.

5.5 Cooperative Target Firmware Architecture

The cooperative-target firmware should be structured as an FSM with explicit states for the bidirectional path:

[**Figure placeholder**]

`Images/BFA.png`

Figure 1: Cooperative-target firmware state machine. Target A: `INIT`→`CONFIG`→`IDLE`→`BUILD_DATA`→`TX_DATA`→`WAIT_ACK`→`LOG`→`IDLE`, with a `PAUSED` branch entered from any state on PAUSE reception. Target B: same skeleton with `RX_LISTEN` replacing the scheduled-IDLE wait, and a `BUILD_ACK`→`TX_ACK` branch on each successful DATA reception.

5.6 Validation: Two-Target Ping-Pong with PAUSE Behavioral Response

Exercise: Bidirectional Ping-Pong Operation

Configure Target A and Target B with the parameters below and verify correct operation:

Parameter	Target A	Target B
Beacon ID	`0x01`	`0x02`
Role	Initiator	Responder
TX Power	+14 dBm	+14 dBm
Spreading Factor	SF7	SF7
Initiator schedule	Fixed, 1 s	— (event-driven on DATA RX)

Using the interceptor (Threat) firmware as a passive sniffer, capture 60 seconds of traffic and verify:

1. Each Target A DATA (`Beacon ID` = `0x01`, `pkt_type` = DATA) is followed by a Target B ACK (`Beacon ID` = `0x02`, `pkt_type` = ACK) whose `Sequence Number` matches the DATA's.
2. Sequence numbers are monotonically increasing on Target A's DATA stream.
3. No CRC failures occur under the fixed-interval schedule.
4. Round-trip time (DATA TX timestamp to ACK RX timestamp on Target A) is bounded (typically <120 ms at SF7 for a 32-byte packet).

Exercise: PAUSE Behavioral Response

With the ping-pong above running, hold the user button (B1) on the interceptor (Threat) board and verify:

1. Target A's USART2 log prints `"I'm Jammed"` within one PAUSE cadence interval (100 ms) of the button being pressed.
2. Target A's DATA transmissions stop while the button is held.
3. Target A resumes normal DATA transmissions within ($\sim$500 ms $+T_{\text{slot}}$) of the button being released.
4. Target B's log does not print `"I'm Jammed"` (the Threat targets the higher-RSSI station, which at equal-distance is whichever is closer to the Threat antenna; if both are equidistant, the choice may alternate — this is a measurement, not a fault).

Session 3 Deliverable

Cooperative Target Subsystem Design Document — A document containing:

1. Packet format specification with field descriptions, byte layout diagram, and the packed `Modulation Code` bit-field decoded.
2. Initiator scheduling design (both TDMA and ALOHA modes) with timing equations.
3. Cooperative target firmware architecture: FSM diagram showing the `WAIT_ACK` branch (Target A), the `BUILD_ACK`→`TX_ACK` branch (Target B), and the shared `PAUSED` branch.
4. Two-target ping-pong validation results: 60-second packet captures showing matched DATA/ACK sequence numbers, round-trip latency CSV, and `"I'm Jammed"` response/recovery log.

6 Session 4: Interceptor (Threat) Subsystem Design

6.1 Objectives

- Design the interceptor data path from the RF module through the STM32 to the FPGA.
- Implement passive capture of every DATA, ACK, and PAUSE frame on the shared channel.
- Implement the per-Beacon-ID RSSI observation table and the button-gated PAUSE TX scheduler that targets the highest-RSSI non-stale station.
- Define the interface protocol between the interceptor MCU and the Nexys A7 FPGA.
- Validate end-to-end data flow from cooperative-target transmission through interceptor capture to FPGA ingestion, plus the closed-loop PAUSE behavioral response on the targets.

6.2 Interceptor Architecture

The interceptor ("Threat") station uses a third STM32 Nucleo-L476RG paired with an RFM95W module, hardware-symmetric with the two cooperative targets but operated under a different firmware role. Unlike the targets, the interceptor handles packets from multiple asynchronous senders without ever participating in the DATA/ACK exchange. Its responsibilities are:

1. **Packet Reception:** Detect and demodulate every incoming DATA, ACK, and PAUSE packet using the SX1276's internal packet engine and `DIO0` interrupt line in continuous-RX mode.
2. **Metadata Extraction:** Read hardware-reported RSSI, SNR, and frequency error from the SX1276 registers for each received packet.
3. **Timestamping:** Capture the STM32 system tick at the moment the `RxDone` interrupt fires, providing an interceptor-side timestamp independent of the sender's claimed timestamp.
4. **RSSI Observation Table:** Maintain a 256-entry per-Beacon-ID table of {`int8_t rssi_dbm, uint32_t last_seen_ms`} updated in the ISR alongside the ring-buffer push. Entries older than a configurable staleness threshold (5 s default) are excluded from highest-RSSI lookups.
5. **Forwarding:** Package the received packet plus interceptor-side metadata into a 43-byte structured frame and transmit it to the FPGA over UART.
6. **Button-Gated PAUSE TX:** While the user button (B1, `PC13`, active-low) is held, every ∼100 ms scan the RSSI table, pick the non-stale Beacon ID with the highest RSSI, build a PAUSE packet targeting that ID with hold-off duration `0x05` (500 ms), and transmit. The Threat's own PAUSE TX is logged on USART2 but *not* forwarded to the FPGA; the dataset captures only what the interceptor receives off-air, which intentionally usually does not include the Threat's own transmissions (own-transmit is shadowed by the half-duplex switch).

Table 6: Interface options for interceptor–FPGA communication.

Interface	Throughput	Complexity	Trade-offs
UART	$\leq$ 921.6 kbps	Low	Simple two-wire connection; sufficient for low station counts; well-supported in both STM32 HAL and VHDL.
SPI (MCU as master)	$\leq$ 10 Mbps	Medium	Higher throughput for scaling; requires clock, MOSI, MISO, and CS lines; FPGA acts as SPI slave.

6.3 Interceptor-to-FPGA Interface

6.3.1 Interface Selection

Recommendation

Begin with UART at 115200 baud for initial integration. At 32 bytes per radio packet wrapped into the 43-byte forwarding frame, UART supports approximately 28 frames per second — comfortably above the operating envelope of the two-target ping-pong configuration plus interleaved PAUSE captures. Transition to SPI only when scaling beyond three concurrent stations or when latency requirements tighten in later modules.

6.3.2 Interceptor-to-FPGA Frame Format

The interceptor wraps every received radio packet (DATA, ACK, or PAUSE) with additional metadata before forwarding to the FPGA. The forwarding-frame layout is unchanged from the original receiver design — the FPGA pipeline (Module 2) does not need to know what the radio packet's type is, only that it received 32 bytes of packet content with valid CRC.

Table 7: Interceptor-to-FPGA forwarding frame (43 bytes).

Field	Offset	Size	Description
Start Delimiter	0	1 B	Fixed byte `0x7E` for frame synchronization.
Frame Length	1	1 B	Total frame length excluding start delimiter and CRC; always `0x29` (41 decimal).
RX Timestamp	2	4 B	Interceptor-side system tick (ms) at `RxDone` interrupt.
RSSI	6	1 B	Received signal strength (dBm, two's complement).
SNR	7	1 B	Signal-to-noise ratio (dB, signed, 0.25 dB resolution).
Freq. Error	8	2 B	Frequency error estimate from SX1276 (Hz, signed 16-bit).
Radio Packet	10	32 B	Complete 32-byte radio packet as received: DATA, ACK, or PAUSE per Table 5.
Frame CRC-8	42	1 B	CRC over bytes 1–41 for transport integrity.

6.4 Interceptor Firmware Architecture

The interceptor firmware operates in an interrupt-driven architecture with two main producers (RF `RxDone` ISR and the button-poll PAUSE scheduler) and one main consumer (the UART forwarder in the main loop):

[Figure placeholder]

`Images/RFA.png`

Figure 2: Interceptor firmware architecture. The `DIO0 RxDone` ISR captures packets into a ring buffer and updates the per-Beacon-ID RSSI table; the main loop drains the ring buffer and forwards 43-byte frames to the FPGA via UART DMA. A separate main-loop poll of `B1` drives the PAUSE TX scheduler at ∼100 ms cadence whenever the button is held.

Concurrency Considerations

- The ring buffer must be sized to handle burst arrivals. With two targets in 1 s ping-pong (effectively two frames per second) plus PAUSE bursts at 10 Hz while the button is held, a 16-entry ring (640 B of beacon structs) is comfortable.
- Use `volatile` qualifiers and a `__DMB()` memory barrier between the buffer write and the head increment to prevent reordering between the ISR producer and main-loop consumer on Cortex-M4.
- Do *not* call HAL `printf` from inside the ISR — it blocks on UART transmit and will drop RF packets. All `printf` debug output happens from the main loop after dequeue.
- The PAUSE TX path must serialize with the receive path: switching the radio into `TX` mode for a PAUSE blocks reception for ∼50 ms (the SF7 airtime of a 32-byte packet). At 10 Hz this consumes ∼50% of the channel time — acceptable while the button is held, but a deliberate measurement consideration.

6.5 FPGA Ingestion Interface

On the FPGA side, the interceptor data is captured by a simple UART receiver module (or SPI slave, if that interface is selected). The Session 4 scope is limited to:

1. **UART Receiver IP:** Implement or instantiate a UART RX module on the Nexys A7 that receives bytes at the configured baud rate and writes them to a small FIFO.

2. **Frame Detection:** Implement start-delimiter detection (`0x7E`) and frame-length extraction to identify complete frames in the byte stream.

3. **Verification:** Display received Beacon IDs on the Nexys A7 seven-segment display or LEDs, confirming end-to-end data flow from cooperative target through interceptor to FPGA fabric.

Exercise: End-to-End Data Flow Validation

With Target A and Target B running the ping-pong configuration from Session 3:

1. Verify that the FPGA receives frames from both Beacon IDs by displaying alternating Beacon IDs on the seven-segment display (`01` and `02` interleaving).

2. Compare the interceptor-side RSSI values displayed on the FPGA with the values logged by the STM32 over USART2 debug to confirm data integrity across the interface.

3. Hold the user button on the interceptor and verify the seven-segment display occasionally shows `FF` (the Threat's own Beacon ID), or alternatively that the `"I'm Jammed"` log appears on the targeted Target's USART2 within 100 ms of button press.

4. Introduce a deliberate CRC error (corrupt one byte in the interceptor firmware before forwarding) and confirm the FPGA's frame CRC check rejects the corrupted frame.

6.6 System Architecture Diagram

Figure 3 shows the complete subsystem architecture validated by the end of this module.

[Figure placeholder]

`Images/Module1Subsystem.png`

Figure 3: Module 1 subsystem architecture. Solid blocks and arrows indicate components and data paths validated by end of Session 4: bidirectional DATA/ACK between Target A and Target B, passive capture by the interceptor (Threat), button-gated PAUSE TX from the Threat back to the targets, and UART forwarding to the FPGA. The dashed ML Pipeline block indicates a downstream module.

Session 4 Deliverable

Interceptor Subsystem Design Document — A document containing:

1. Interceptor architecture description and data flow diagram.
2. Interceptor-to-FPGA interface selection rationale and frame format specification.
3. Interceptor firmware design: interrupt architecture, ring buffer sizing, RSSI observation table, button-gated PAUSE TX scheduler with cadence and hold-off rationale.
4. FPGA ingestion module description (UART RX, frame detection, verification outputs).
5. End-to-end validation test results: cooperative-target ping-pong → interceptor → FPGA data flow confirmation, and PAUSE behavioral response measurements (response and recovery latency).

7 Hardware Platform Reference

Table 8: Required hardware components for Module 1.

Component	Qty	Purpose
STM32 Nucleo-L476RG	≥ 3	Two cooperative-target MCUs (Target A and Target B) + one interceptor (Threat) MCU
RF Transceiver Module	≥ 3	Sub-GHz LoRa communication (RFM95W, 906.5 MHz operating carrier in the U.S. 915 MHz ISM band)
915 MHz Spring Antenna	≥ 3	Required for RF transmission/reception
Digilent Nexys A7-100T	1	FPGA development board (Artix-7, 100T)
USB Micro-B Cables	4–5	Power and programming for Nucleo and Nexys boards
Breadboard + Jumper Wires	—	Prototyping connections (SPI, GPIO, power, UART)
Logic Analyzer (optional)	1	SPI/UART bus debugging and timing verification

8 Assessment Rubric

Table 9: Module 1 assessment rubric.

Assessment Item	Weight	Due
RF Module Trade Study Report (weighted matrix, link budget, justification)	20%	Session 1
Hardware Requirements Document (BOM, interfaces, power budget, validation evidence)	20%	Session 2
Cooperative Target Subsystem Design (packet format with packed type field, ping-pong firmware, PAUSE listener, two-target tests)	30%	Session 3
Interceptor Subsystem Design (data path, RSSI table, PAUSE scheduler, end-to-end + PAUSE behavioral validation)	30%	Session 4

9 Recommended Resources

9.1 Datasheets and Technical References

1. Semtech, "SX1276/77/78/79 Datasheet," Rev. 7, 2020.
2. STMicroelectronics, "STM32L476xx Reference Manual," RM0351, Rev. 9, 2023.
3. Xilinx/AMD, "Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181, 2023.
4. Digilent, "Nexys A7 Reference Manual," Rev. C, 2022.

9.2 Application Notes

1. Semtech AN1200.22, "LoRa Modulation Basics."
2. Semtech AN1200.13, "SX1276/7/8 LoRa Modem Design Guide."
3. STMicroelectronics AN4286, "SPI Protocol Used in the STM32 Bootloader."
4. STMicroelectronics AN4013, "STM32 Timer Overview and Low-Power Timer Usage."

9.3 Textbooks

1. P. P. Chu, *FPGA Prototyping by VHDL Examples: Xilinx Artix-7 Edition*, 2nd ed., Wiley, 2017.
2. R. E. Haskell and D. M. Hanna, *Digital Design Using Digilent FPGA Boards: VHDL / Vivado Edition*, LBE Books, 2019.
3. C. Ünsalan and B. Tar, *Digital System Design with FPGA: Implementation Using Verilog and VHDL*, McGraw-Hill, 2017.
4. B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed., Prentice Hall, 2001.
5. J. W. Valvano, *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers*, 6th ed., 2023.

10 Transition to Module 2

Upon successful completion of Module 1, students will have:

- A validated hardware platform with confirmed SPI communication across all three nodes.
- A fully specified and tested cooperative-target subsystem (Target A and Target B) capable of bidirectional ping-pong operation with PAUSE behavioral response.
- An interceptor (Threat) subsystem that passively captures DATA, ACK, and PAUSE traffic, maintains a per-Beacon-ID RSSI observation table, and operates the button-gated PAUSE TX scheduler.
- End-to-end data flow from cooperative-target transmission through interceptor to FPGA frame ingestion.
- Comprehensive design documentation covering the 32-byte packet format with packed DATA/ACK/PAUSE type field, the 43-byte forwarding frame, the firmware architectures for both cooperative-target and interceptor roles, and the closed-loop intercept-and-act behavior of the PAUSE protocol.

Module 2 will build directly on these foundations with FPGA RTL design and signal processing simulation. The UART/SPI ingestion interface validated in Session 4 becomes the input to the FPGA processing pipeline; the 32-byte radio packet format defines the data structures that the RTL must parse; and the packed `Modulation Code` byte's low two bits become the `pkt_type` field in the Module 2 feature vector, classified at the data-to-model interface in Module 4.

Glossary

ASK	Amplitude Shift Keying	1
BOM	Bill of Materials	9, 22
CLT	Command Line Tools (STM32Cube)	7
CRC	Cyclic Redundancy Check	4, 8, 11, 15, 18, 20
CSS	Chirp Spread Spectrum	1, 4
FIFO	First In, First Out	4, 19
FPGA	Field-Programmable Gate Array	1, 2, 3, 6, 9, 10, 13, 17, 18, 19, 20, 21, 22, 23, 24
FSK	Frequency Shift Keying	1, 4, 11
FSM	Finite State Machine	13, 14, 16
GPIO	General-Purpose Input/Output	1, 8, 9, 22
HAL	Hardware Abstraction Layer	6, 7, 13, 18, 19
I^2C	Inter-Integrated Circuit	1
IDE	Integrated Development Environment	7
ILA	Integrated Logic Analyzer	9
MCU	Microcontroller Unit	7, 8, 17, 18, 22
ML	Machine Learning	1, 13, 20
OOK	On-Off Keying	4, 11
RF	Radio Frequency	1, 3, 4, 5, 6, 8, 9, 17, 19, 22
RSSI	Received Signal Strength Indicator	7, 8, 9, 17, 18, 19, 20, 24
RTL	Register-Transfer Level	1, 24
SNR	Signal-to-Noise Ratio	7, 8, 9, 17, 18
SPI	Serial Peripheral Interface	1, 3, 4, 5, 6, 7, 8, 9, 18, 19, 22, 23, 24
UART	Universal Asynchronous Receiver/Transmitter	1, 5, 9, 17, 18, 19, 21, 22, 24
VHDL	VHSIC Hardware Description Language	1, 2, 18, 23

End of Module 1 Lesson Plan